

Computer Architecture: Weird Machines, Rust, *The End*

Dr. Colin O'Flynn

Lectures

- How does a program run on a machine?
- How do you compile from high-level code to machine code?
- How does the machine run? (Control Flow)
- How does the machine store, access, and control data?
- How does the machine execute instructions efficiently?
- **What other machines & compilers are out there?**

Very long instruction word (VLIW)

- Instruction words contain everything to do at once
 - Unlike with superscalar where CPU decides what it can do, or re-orders pipeline
- Motivation: the scheduling logic is complicated, why do this in hardware? Make the compiler figure it out!
- E.g., the following is one (48-bit) instruction:

```
f12 = f0 * f4, f8 = f8 + f12, f0 = dm(i0, m3), f4 = pm(i8, m9);
```

Super Harvard Architecture Single-Chip Computer (SHARC)

SHARC Processor Architectural Overview

Super Harvard Architecture

Analog Devices' 32-Bit Floating-Point SHARC® Processors are based on a Super Harvard architecture that balances exceptional core and memory performance with outstanding I/O throughput capabilities. This "Super" Harvard architecture extends the original concepts of separate program and data memory busses by adding an I/O processor with its associated dedicated busses. In addition to satisfying the demands of the most computationally intensive, real-time signal-processing applications, SHARC processors integrate large memory arrays and application-specific peripherals designed to simplify product development and reduce time to market.

The SHARC processor portfolio currently consists of four generations of products providing code-compatible solutions ranging from entry-level products priced at less than \$10 to the highest performance products offering fixed- and floating-point computational power to 450 MHz/2700 MFLOPs. Irrespective of the specific product choice, all SHARC processors provide a common set of features and functionality useable across many signal processing markets and applications. This baseline functionality enables the SHARC user to leverage legacy code and design experience while transitioning to higher-performance, more highly integrated SHARC products.

Common Architectural Features

- 32/40-Bit IEEE Floating-Point Math
- 32-Bit Fixed-Point Multipliers with 64-Bit Product & 80-Bit Accumulation
- No Arithmetic Pipeline; All Computations Are Single-Cycle
- Circular Buffer Addressing Supported in Hardware
- 32 Address Pointers Support 32 Circular Buffers
- Six Nested Levels of Zero-Overhead Looping in Hardware
- Rich, Algebraic Assembly Language Syntax
- Instruction Set Supports Conditional Arithmetic, Bit Manipulation, Divide & Square Root, Bit Field Deposit and Extract
- DMA Allows Zero-Overhead Background Transfers at Full Clock Rate Without Processor Intervention

First Generation SHARC products offer performance to 66 MHz/ 198 MFLOPs and form the cornerstone of the SHARC processor family. Their easy-to-use Instruction Set Architecture that supports both 32-bit fixed-point and 32/40-bit floating data formats combined with large memory arrays and sophisticated communications ports make

<https://www.analog.com/en/lp/001/sharc-processor-architectural-overview.html>

SHARC Features & Oddities (designed ~1994)

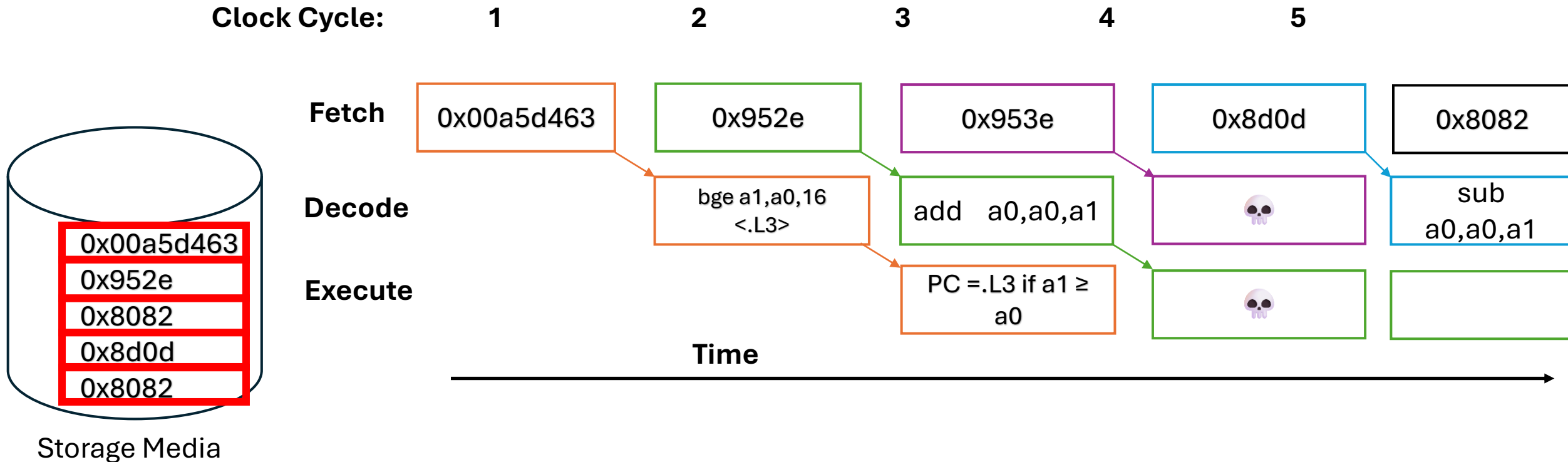
- Two register banks (context switching)
- Word based addressing (not byte-based)
- Instructions can be conditionally executed
- Built-in loop support (more on that)
- After a branch instruction the next two instructions can be executed (“delay slots”)

Delay Slots

- Remember the pipeline problem – why flush away those cycles?
- Instead we execute the instructions anyway
- Requires us to think about what instructions we want to execute:

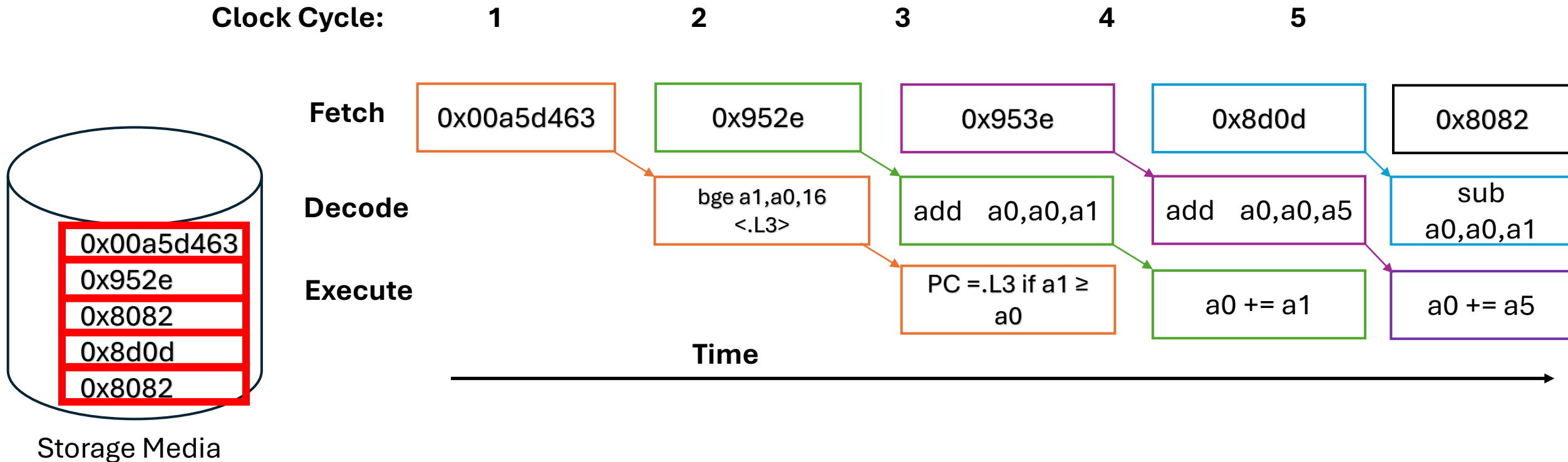
```
loop_start:
    add r0, #1
    ldr r2, [r0, #8000]
    str r2, [r0, #8000]
    cmp r0, #5
    bne loop_start
```

Remember: Pipeline with Branch FLUSH



*NOTE: in RISC-V **a0/a1**/etc are general-purpose registers.*

Delay Slots: Just Execute Them!!



*NOTE: in RISC-V **a0/a1**/etc are general-purpose registers.*

Delay Slots

```
loop_start:
    add r0, #1
    ldr r2, [r0, #8000]
    str r2, [r0, #8000]
    cmp r0, #5
    bne loop_start
```

```
loop_start:
    add r0, #1
    cmp r0, #5
    bne loop_start
    ldr r2, [r0, #8000]
    str r2, [r0, #8000]
```

Fibonacci Example with SHARC

```
// Calculate the Fibonacci number Fn,  
n need to be at least 2  
int fibonacci(int n) {  
    int a = 0, b = 1, f;  
    for (int i = 1; i < n; i++) {  
        f = a + b;  
        a = b;  
        b = f;  
    }  
    return f;  
}
```

```
        r2 = 1; // loop counter  
        r8 = 0;  
        r12 = 1;  
loop_start:  
        r0 = r8 + r12;  
        r8 = r12;  
        r12 = r0;  
        r2 = r2 + 1;  
        comp(r2, r4);  
        if lt jump loop_start;
```

https://zephyray.me/post/sharc_tutorial_2/

8 cycles per iteration (6 instructions + 2 stalls)

Adding Delay Slots

```
    r2 = 1; // loop counter
    r8 = 0;
    r12 = 1;
loop_start:
    r0 = r8 + r12;
    r2 = r2 + 1;
    comp(r2, r4);
    if lt jump loop_start (db);
    r8 = r12;
    r12 = r0;
```

6cycles per iteration (no stalls)

Using Loop Counter

```
    r8 = 0;  
    r12 = 1;  
    r4 = r4 - 1;  
lcntr = r4, do loop_end until lce;  
    r0 = r8 + r12;  
    r8 = r12;  
loop_end:  
    r12 = r0;
```

3 cycles per iteration

Using VLIW Multi-Issue

```
    r8 = 0;  
    r0 = 1;  
    r4 = r4 - 1;  
lcntr = r4, do loop_end until lce;  
loop_end:  
    r0 = r8 + r0, r8 = r0;
```

1 cycles per iteration

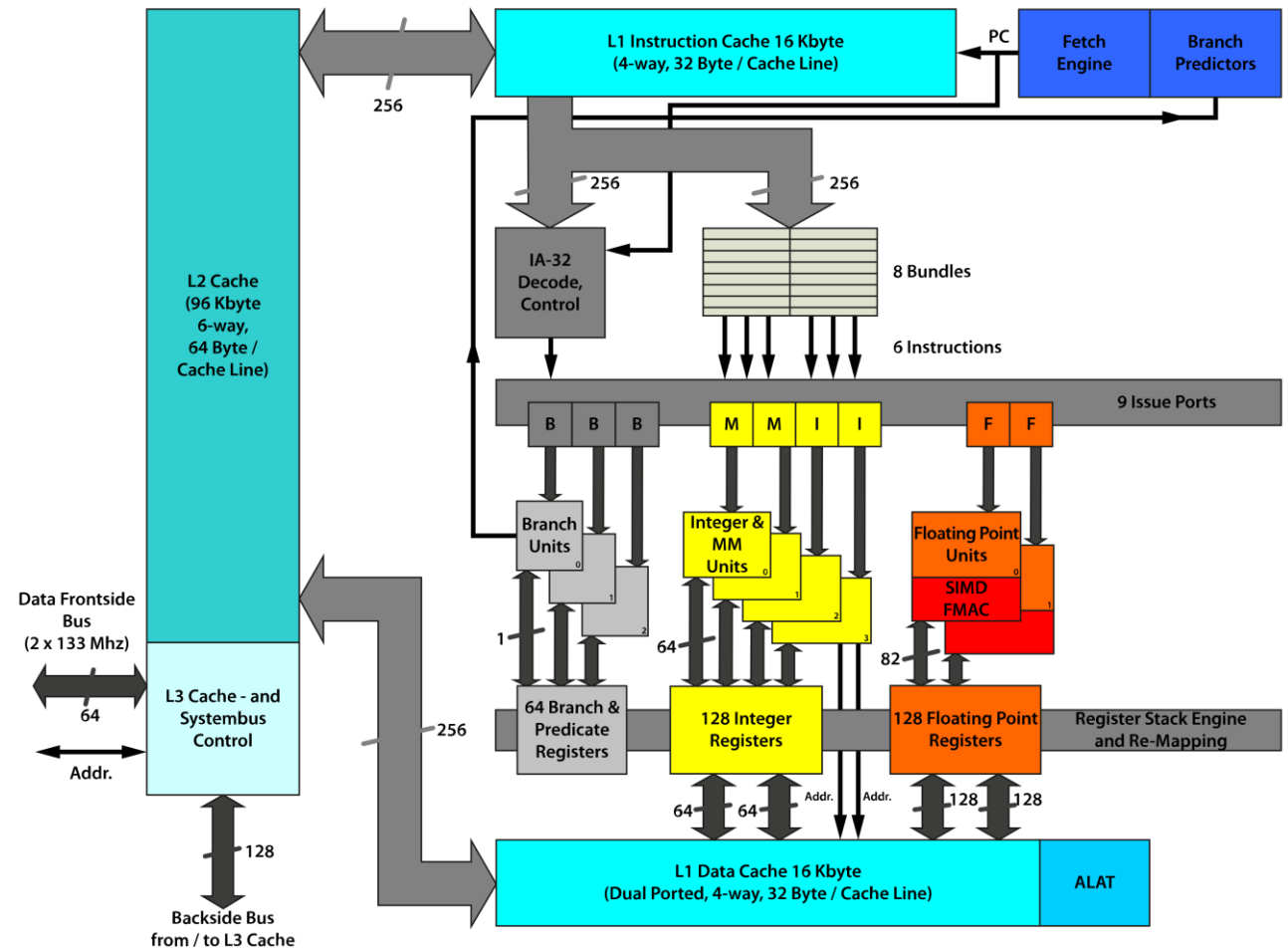
SHARC Usage

- Was used extensively in RADAR and other processing applications
- There have been many Digital Signal Processing (DSP) specific architectures in history as that is a more unique challenge
- Nowadays you might think of GPU architectures as being similar

SHARC is perhaps a “weird” machine but not a failure. Next...

Intel Itanium (“Itanic”) (2001 – 2020)

- IA-64: One of the original 64-bit architectures
- VLIW → Explicitly parallel instruction computing (EPIC)
- The compiler decides what happens in parallel
 - In “SuperScalar” the CPU decides



By Fred the Oyster, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=35228767>

Itanium Failures

- Legacy SW was not compatible
- Compiler parallelism wasn't as great as thought
- x86-64 shortly arrived – which *was* compatible
 - And these cores kept getting better



Itanium sales fall \$13.4bn shy of \$14bn forecast

Sun top Opteron vendor

 [Ashlee Vance](#)

Mon 30 Aug 2004 // 18:37 UTC



Itanium server sales are coming in just as expected, hitting the \$14bn revenue mark halfway through 2004.

Okay, well, not quite \$14bn. That's the total analyst powerhouse IDC had once predicted Itanium would reach by the midway point of this year. In actual fact, total Itanium sales have hit \$606m through the first two quarters of this year. Other organs might mock a \$13.4bn miss by one of the world's leading number crunching firms but not us. We'll let you come to your own conclusions about such an incredible gaffe.

The \$606m figure arrives courtesy of Gartner, which just released its second quarter server sales data. We covered the basics last week but thought a break down of Itanium and Opteron sales would provide a bit more insight into the server market. So here goes.

https://www.theregister.com/2004/08/30/opteron_itanium_sales_q2/

Intel i860 (1980 – mid 1990s)

- First million transistor chip
- Allowed execution of core (ALU) + floating point instructions simultaneously
- VLIW support required for parallel execution



Compiler Problems

- High performance should be possible (measured with “Million Floating-point Operations Per Second” = MFLOPS)
- Expected up to 80 MFLOPS
- ..most real programs in assembly (high effort) got ~40 MFLOPS
- ..compilers even worse ~10 MFLOPS

Compatibility is King!

- In both of these cases – the *more popular existing systems won*
- x86 instruction set was so widely used, compilers for x86 much more efficient than the "bespoke" compilers
- So....

Transmeta's Crusoe

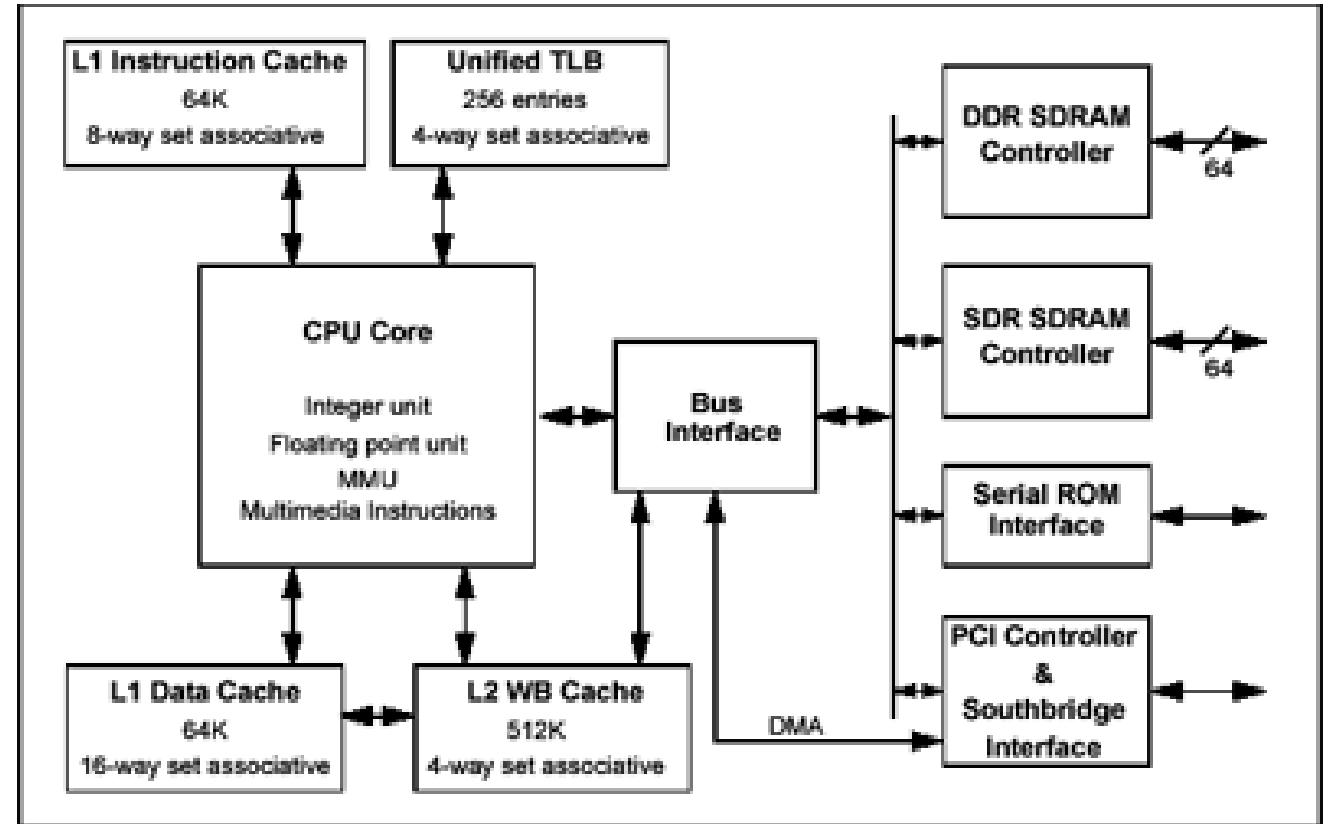
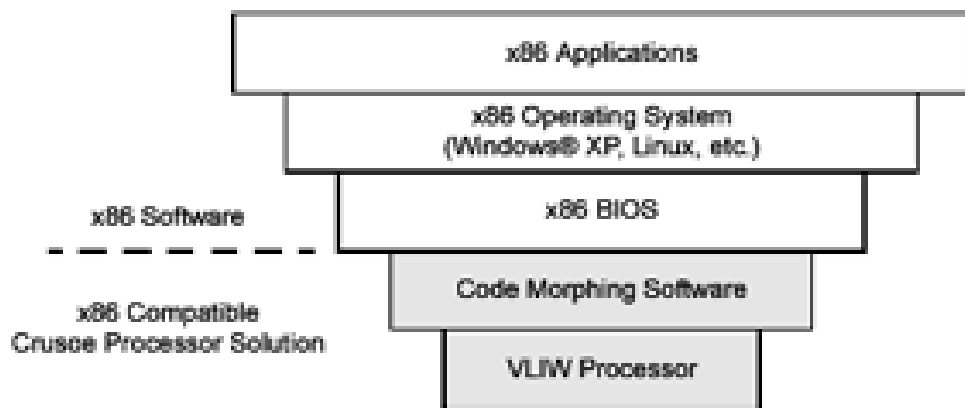
- Let's build an x86 conversion layer... into our chip!



- Targeted low-power applications at a time when mobile wasn't very common
- Many interesting developments that did get used in the future
- But what were they doing?

<https://web.archive.org/web/20040701093906/http://www.transmeta.com/>

Transmeta's Crusoe

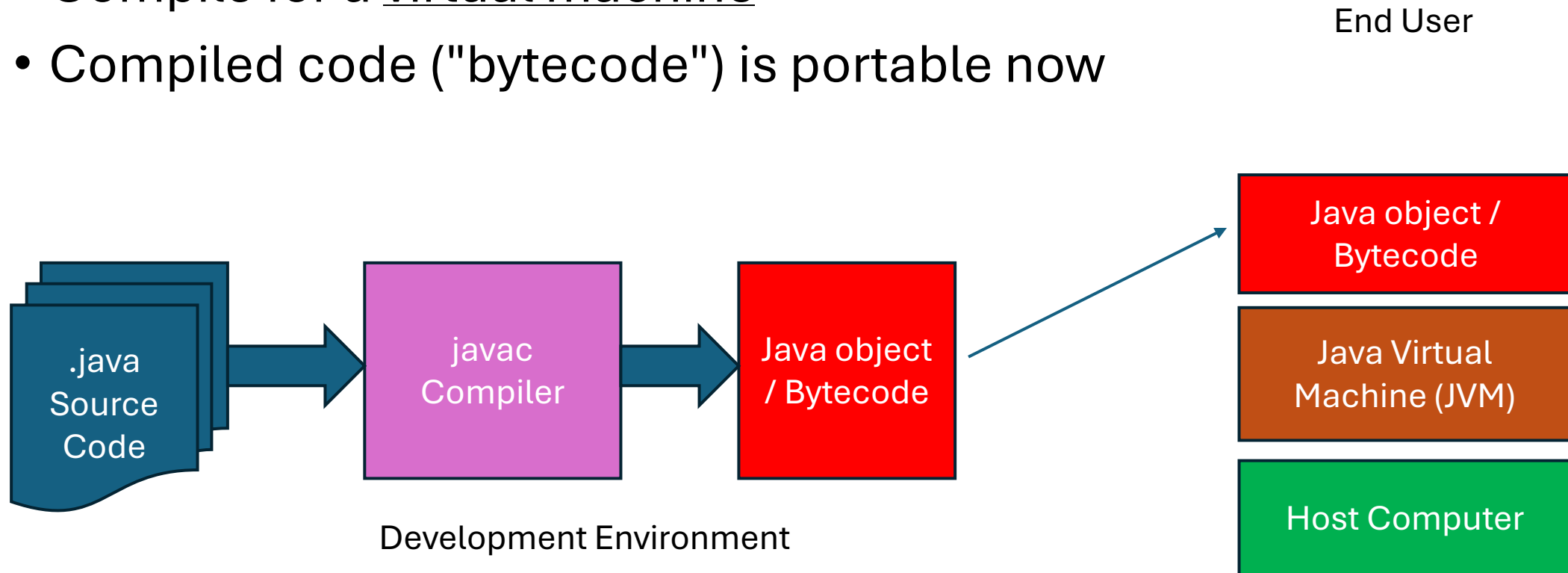


Transmeta LongRun Technology

- Adjusted CPU clock frequency & voltage dynamically for workload
 - Dynamic Voltage and Frequency Scaling (DVFS)
- This technology is used extensively in current processors

Java & The Java Virtual Machine (JVM)

- Compile for a virtual machine
- Compiled code ("bytecode") is portable now



e.g., Java Bytecode

```
int add(int a, int b) {  
    return a + b;  
}
```



```
public int add(int, int);
```

Code:

0: iload_1

1: iload_2

2: iadd

3: ireturn

https://foojay.io/today/java-bytecode-simplified-journey-to-the-wonderland-part-1/#:~:text=For%20example%2C%20if%20we%20want,%2D%200:%20iconst_3%201:%20ireturn

Java – But Why?

- Java was a very popular language for larger embedded systems (more "app-like", smart things, DVD players, etc)
 - Consider the ecosystem – lots of different processor architectures, Java meant you had portability across them (and for anything that comes later)
 - There have even been Java processors that implement the JVM in hardware
 - As defacto standards (Arm, RISC-V) appeared in embedded, Java has become less popular.
- Java is still popular in many spaces, included Android app development

Intel iAPX 432 – Intel's First Failure



- We looked at this earlier – it had *object oriented* hardware implementation that fundamentally made code "safe"
- Performance was simply too low – at a time when performance was #1 consideration
- Initially only worked / supported with Ada compilers

Ada Programming Language

- Created for embedded & real-time systems
- Designed to “fundamentally solve” many of the problems that plagued (*and still plague*) embedded systems:
 - Memory leaks
 - Pointer issues
 - Crashes due to overwriting buffers
- Uses strongly typed language features
- Pushed by US Military / Government as “The Future of Computer Programming”
 - Mandated for use in new software for US Military in 1991 (*mostly removed by 1997*)

Ada Hello World

```
1  with Ada.Text_IO; use Ada.Text_IO;  
2  procedure Hello is  
3  begin  
4      Put_Line ("Hello, world!");  
5  end;
```

Ada Types

```
1  type Day_type    is range    1 ..   31;
2  type Month_type  is range    1 ..   12;
3  type Year_type   is range 1800 .. 2100;
4  type Hours is mod 24;
5  type Weekday is (Monday, Tuesday, Wednesday, Thursday, Friday,
Saturday, Sunday);
6
7  type Date is
8      record
9          Day      : Day_type;
10         Month    : Month_type;
11         Year     : Year_type;
12     end record;
```

Ada in 2026

- The goal of making a safer embedded language has long been a noble pursuit
- To wrap this up – let's introduce something useful for you to know.

The Memory Management Problem

Programs need to store data on the heap — memory that outlives a single function call. The hard question is: who decides when to free it?

Manual (C / C++)

Advantages:

- No runtime overhead at all
- Fully predictable timing
- Works on hardware with 2KB RAM

Trade-offs:

- Error-prone: leaks, double-free, use-after-free
- Developer responsible for every allocation

Garbage Collector (Java, Python, Go, C#)

Advantages:

- Automatic — developer never calls `free()`
- Eliminates most memory errors at runtime

Trade-offs:

- Runtime pauses, overhead, non-determinism
- Large memory footprint; needs OS support

Ownership (Rust)

Advantages:

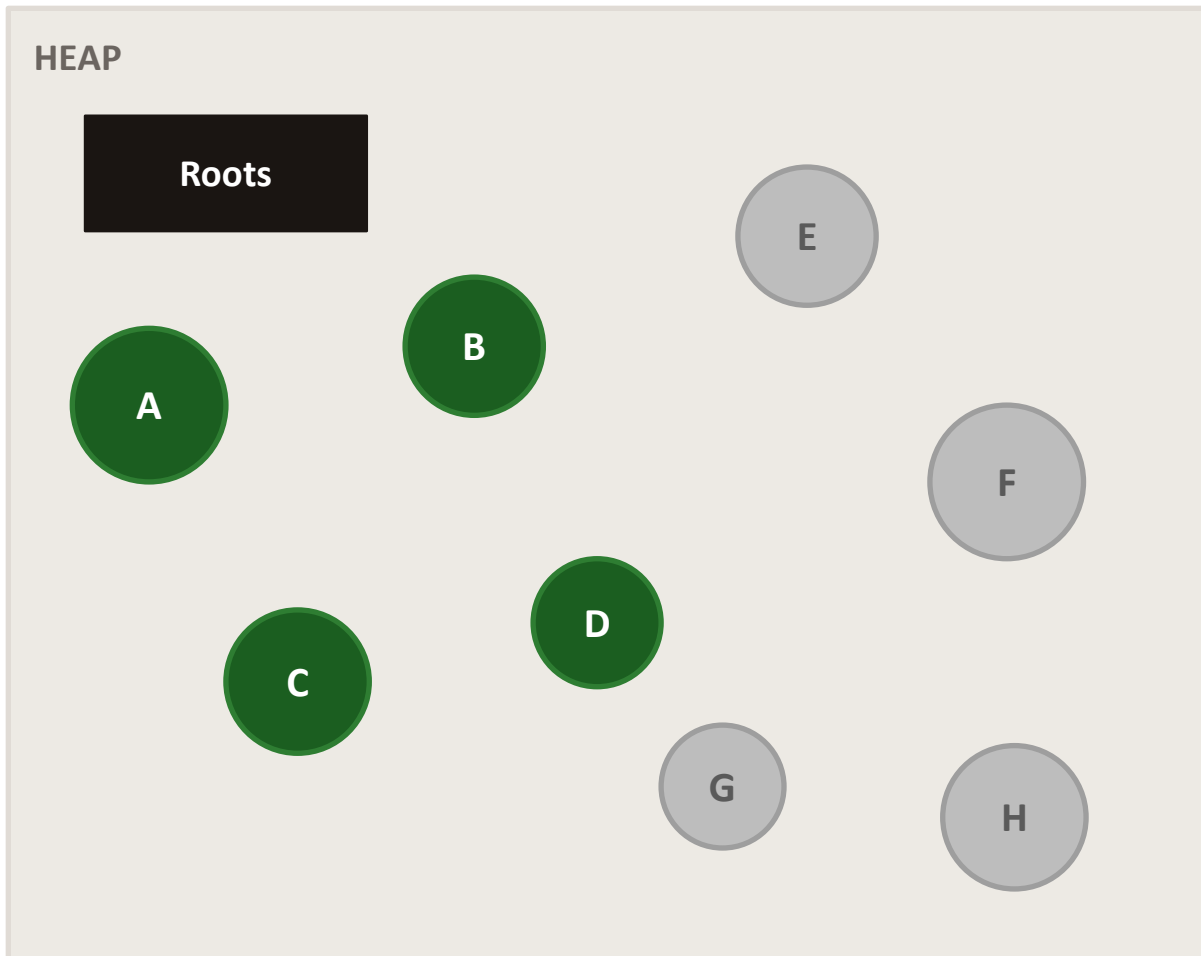
- Automatic — freed when owner goes out of scope
- Zero runtime cost; compile-time guarantees

Trade-offs:

- Steeper initial learning curve

How GC Works: Mark & Sweep

The most common GC algorithm. It runs in two phases — and your program must completely stop while this happens.



Phase 1: MARK

Starting from "root" objects (stack variables, static fields), the GC traverses every reachable reference and marks each visited object as live.



Reachable — marked LIVE



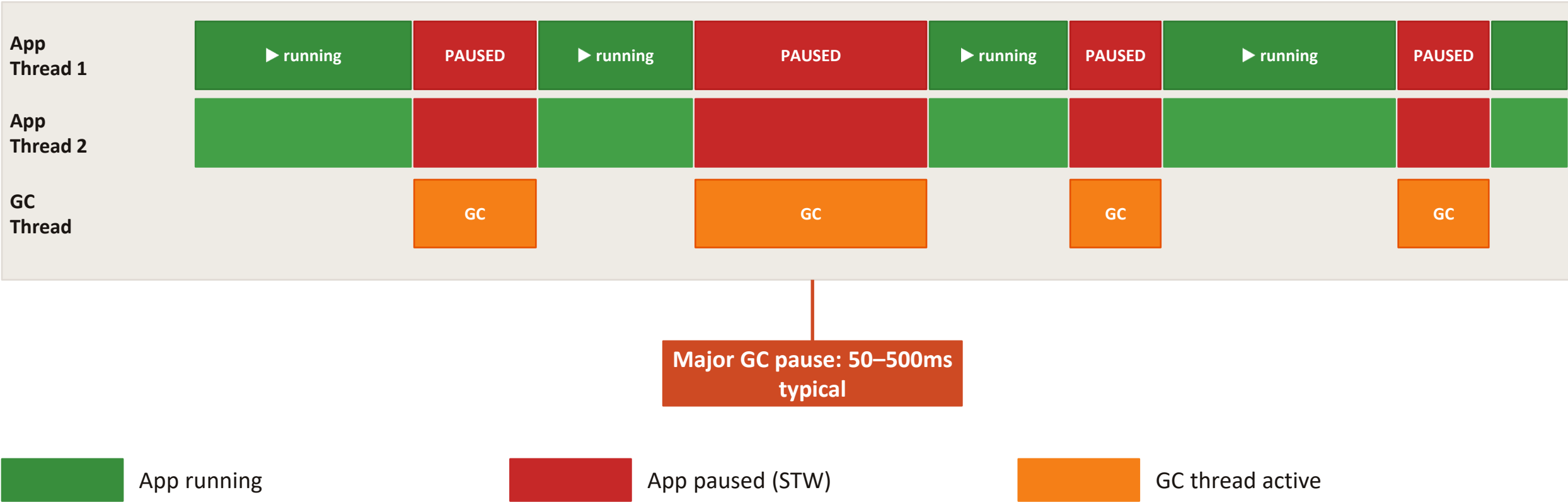
Unreachable — queued for collection

Phase 2: SWEEP

All unmarked objects are freed. The heap reclaims that space. During BOTH phases: your entire program is paused.

The Stop-The-World Pause

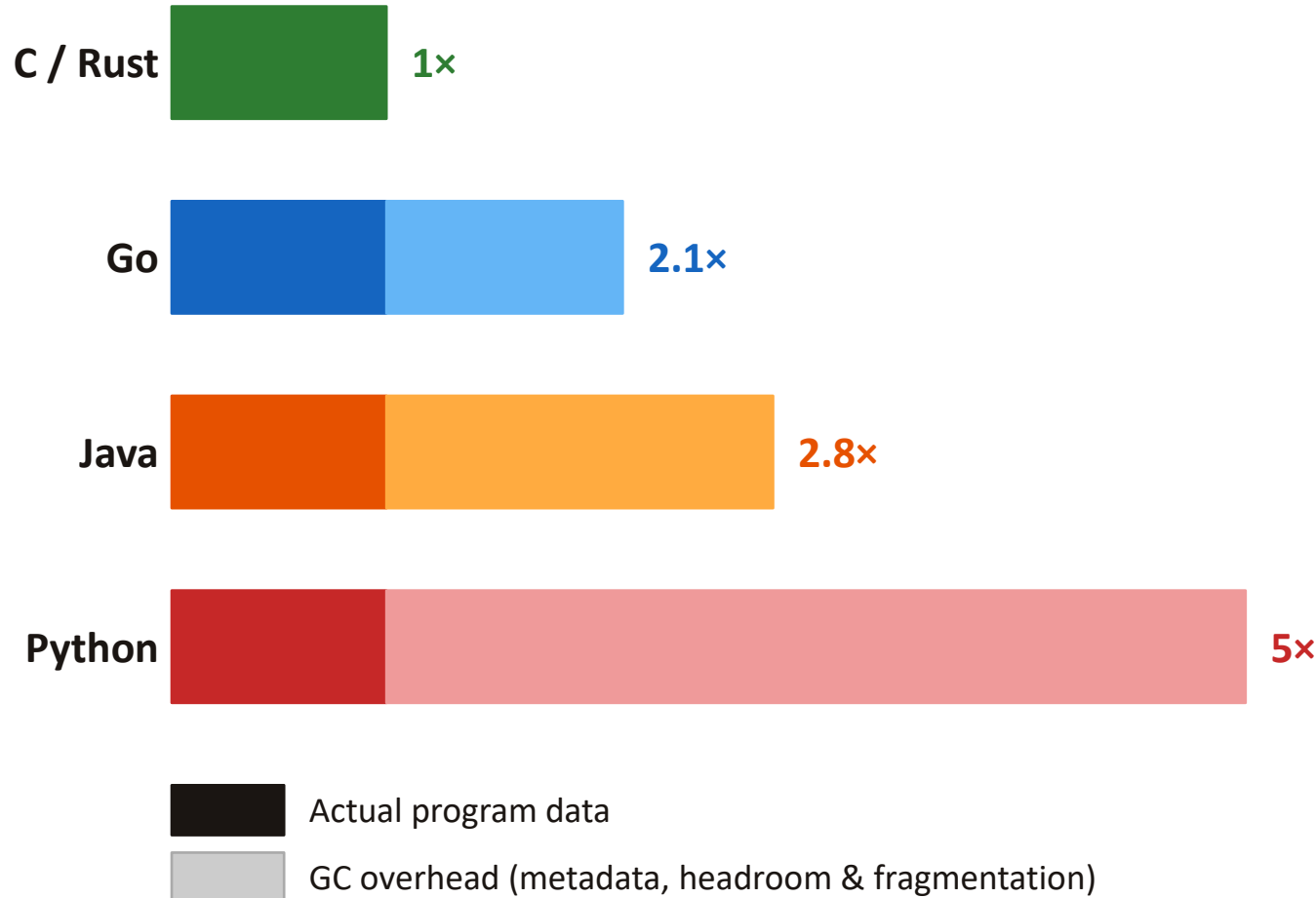
To safely traverse the heap, the GC must freeze ALL application threads. No thread may modify object references while the GC is scanning.



Memory Overhead & Fragmentation

GC-managed runtimes need significantly more memory than your program's actual data — a luxury embedded systems cannot afford.

Relative memory usage (same program, data = 1.0×)



GC Headroom

GC needs 2–3× live data to operate efficiently. At 70%+ heap utilization, performance degrades sharply as GC runs constantly.

Fragmentation

After many alloc/free cycles, free memory becomes scattered in small chunks. The GC must occasionally "compact" the heap — another STW pause.

Object Overhead

Every heap object carries a header (type tag, GC flags, lock word). A single boolean may cost 16–32 bytes including header and alignment.

Non-Determinism: The Real-Time Killer

With a GC, you cannot predict when collection will run or how long it will take. For real-time systems, this unpredictability is catastrophic.

Rust / C — Deterministic response times:



Every response: ~8ms (consistent, predictable)

GC Language — Non-deterministic response times:



Normal: ~8ms. But GC pauses hit without warning: minor ~10ms, major 100–500ms

Real-time systems must guarantee response within a hard deadline. A GC pause that fires at the wrong moment misses that deadline entirely — causing audio glitches, sensor sampling gaps, or control system failures.

Rust's Ownership Model (GC-free)

Rust's ownership system is enforced at compile time — no runtime checks, no GC, zero overhead.

1 Each value has one owner

When the owner goes out of scope, the value is automatically freed — no malloc/free pairs, no memory leaks by design.

2 You can borrow references

Multiple read-only borrows are allowed, OR one exclusive mutable borrow — but never both at once. Data races become impossible.

3 The compiler enforces everything

Invalid borrows, use-after-free, and data races are caught at compile time, not discovered on hardware

```
// 1. Ownership moves on assignment
let v = vec![1, 2, 3]; // v owns the Vec
let w = v;             // ownership moves to w
// println!("{:?}", v); // ✗ compile error!

// 2. Borrowing: share without moving
let s = String::from("hello");
let len = compute_len(&s); // borrow s (read-only)
println!("{}", s, len); // s still valid ✓

// 3. Mutable borrow: exclusive access
let mut x = 5;
let r = &mut x; // exclusive mutable borrow
*r += 1;        // only r can access x here
println!("{}", x); // borrow ended, x accessible ✓
```

Rust vs C: A Quick Comparison

Feature	C	Rust
Memory Safety	Manual (error-prone)	Compile-time enforced
Garbage Collector	None	None
Data Races	Possible at runtime	Impossible (compile-time)
Abstraction Cost	Low	Zero (monomorphized)
Package Manager	None (ad hoc)	Cargo (built-in)
Error Handling	Return codes / errno	Result<T,E> types
null Pointers	Allowed (segfault risk)	Do not exist (Option<T>)
Embedded Support	Decades of ecosystem	Growing fast, practical now

Installing Rust (WSL or Native)

rustup

An installer for the systems programming language [Rust](#)

To install Rust, download and run
[rustup-init.exe](#)
then follow the onscreen instructions.

You may also need the [Visual Studio prerequisites](#).

If you're a Windows Subsystem for Linux user run the following in your terminal, then follow the onscreen instructions to install Rust.

```
$ curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```



You appear to be running Windows 64-bit. If not, [display all supported installers](#).

Need help?
Ask the [community](#)!

Setting Up Your Environment

1 Install Rust

```
# Install rustup – the Rust toolchain manager
curl --proto '=https' --tlsv1.2 -sSf \
  https://sh.rustup.rs | sh

# Verify
rustc --version && cargo --version
```

2 Add Cross-Compilation Targets

```
# Cortex-M4 / M7 with & w/o hardware float (e.g. STM32F3/F4)
rustup target add thumbv7em-none-eabihf thumbv7em-none-eabi

# Cortex-M0 / M0+ (many simpler MCUs)
rustup target add thumbv6m-none-eabi
```

3 Install Flashing Tools + Embedded Tools etc

```
# probe-rs-tools gives us cargo-embed and a bunch of stuff
cargo install probe-rs-tools
```


Example: Blink an LED

The "Hello, World!" of embedded — toggle an LED every 500ms. Target: STM32 Nucleo 303

```
#![no_std]
#![no_main]
use cortex_m::{asm, peripheral::Peripherals as CorePeripherals};
use cortex_m_rt::entry;
use stm32f3xx_hal::{pac, prelude::*};
use panic_halt as _;

#[entry]
fn main() -> ! {
    let dp = pac::Peripherals::take().unwrap();
    let _cp = CorePeripherals::take().unwrap();

    let mut flash = dp.FLASH.constrain();
    let mut rcc = dp.RCC.constrain();
    let _clocks = rcc.cfgr.freeze(&mut flash.acr);

    let mut gpioa = dp.GPIOA.split(&mut rcc.ahb);
    let mut led = gpioa.pa5.into_push_pull_output(
        &mut gpioa.moder,
        &mut gpioa.otyper,
    );

    loop {
        led.set_high().ok();
        asm::delay(8_000_000);
        led.set_low().ok();
        asm::delay(8_000_000);
    }
}
```

`Peripherals::take()` is a one-time call. The type system guarantees you can't accidentally use the same peripheral in two places.

`into_push_pull_output()` returns a typed pin. The type encodes the pin mode — you cannot call `set_high()` on an input pin.

`asm::delay()` blocks using a busy-loop counter, can use a HAL delay function as well.

`loop {}` never exits. Embedded programs run until power is removed — there is no OS to return to.

Example: Send Data Over UART

Print debug messages or sensor readings to a serial terminal — essential for any embedded project.

```
#![no_std]
#![no_main]

use cortex_m_rt::entry;
use stm32f4xx_hal::{pac, prelude::*, serial};
use core::fmt::Write;
use panic_halt as _;

#[entry]
fn main() -> ! {
    let dp = pac::Peripherals::take().unwrap();
    let rcc = dp.RCC.constrain();
    let clocks = rcc.cfgr.sysclk(48.MHz()).freeze();
    let gpioa = dp.GPIOA.split();
    let tx = gpioa.pa2.into_alternate::<7>();
    let rx = gpioa.pa3.into_alternate::<7>();
    let serial = dp.USART2.serial((tx, rx), 115_200.bps(), &clocks).unwrap();
    let (mut tx, _rx) = serial.split();
    for count in 0u32.. {
        writeln!(tx, "count = {}", count).ok();
    }
}
```

Key concepts:

Type-safe pin config

`into_alternate::<7>()` selects UART alternate function at compile time. Wrong number = compiler error before you even flash.

core::fmt::Write

The Write trait provides `writeln!` for any "writer" type. Works just like `println!` but writes to your serial port.

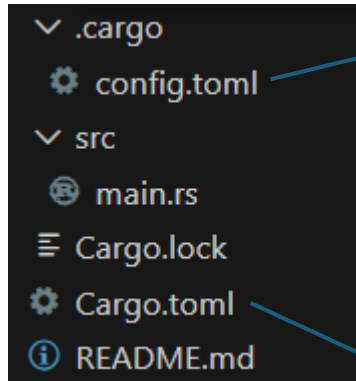
.split()

Separates the serial port into independent TX and RX with their own ownership — prevents misuse.

.ok() vs .unwrap()

`writeln!(...).ok()` silently ignores errors (e.g. TX buffer full). Use `.unwrap()` during debugging to catch issues.

Building Blink For Real: Cargo



```
[build]
target = "thumbv7em-none-eabihf"
```

```
[target.thumbv7em-none-eabihf]
runner = "probe-rs run --chip STM32F303RETx" # optional: flash & run
rustflags = ["-C", "link-arg=-Tlink.x"]
```

```
[package]
name = "blinky"
version = "0.1.0"
edition = "2021"
```

```
[dependencies]
cortex-m-rt = "0.7"
cortex-m    = "0.7"           # needed for SYST-based delay
stm32f3xx-hal = { version = "0.10", features = ["stm32f303xe"] }
panic-halt = "0.2"
```

```
[profile.release]
opt-level = "s" # optimize for size
lto = true
```

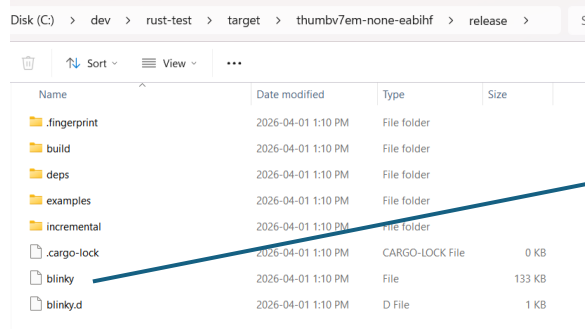
Building Blinky For Real: Cargo

```
colin@COL-PERBOX:/mnt/c/dev/rust-test$ cargo build --release
Compiling autocfg v1.5.0
Compiling proc-macro2 v1.0.106
Compiling quote v1.0.45
Compiling unicode-ident v1.0.24
Compiling semver-parser v0.7.0
```

<REMOVED LONG OUTPUT>

```
Compiling enumset_derive v0.14.0
Compiling enumset v1.1.10
Compiling blinky v0.1.0 (/mnt/c/dev/rust-test)
Finished `release` profile [optimized] target(s) in 11.04s
```

This generates an ELF file in `target\thumbv7em-none-eabihf\release`

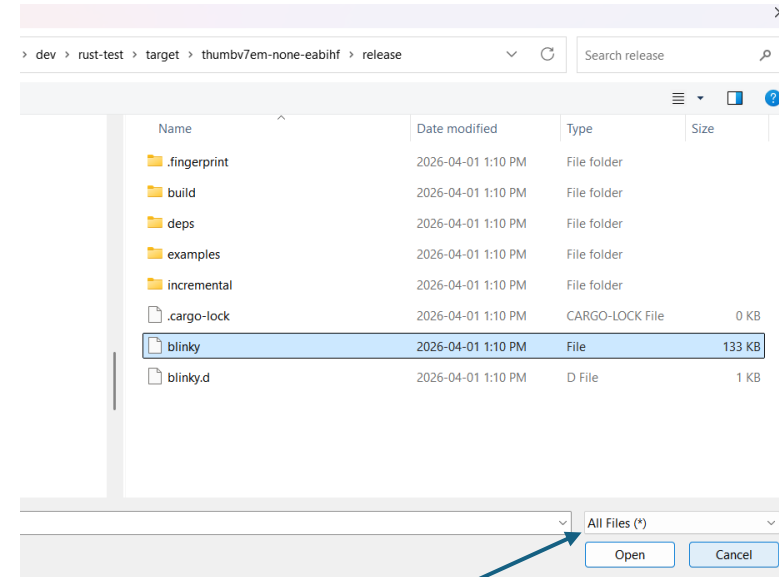
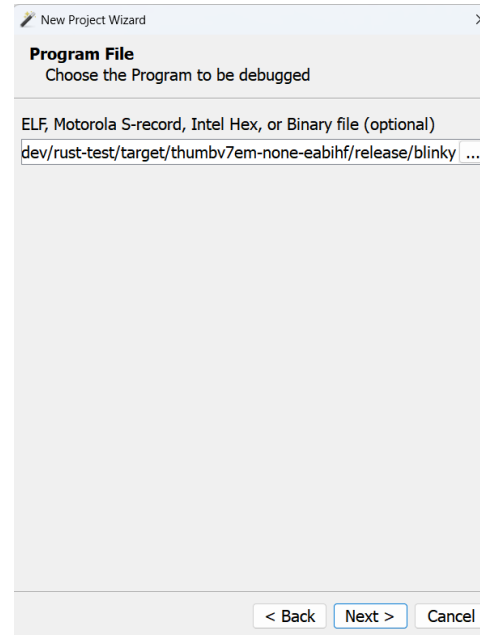
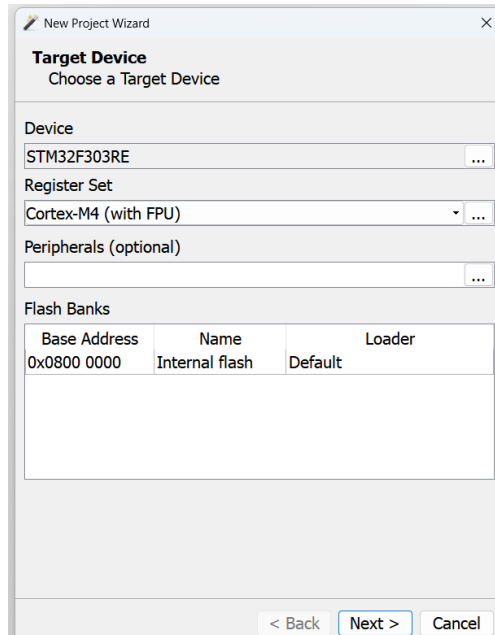


blinky is a .elf file – note NO EXTENSION

Programming Blinky

If you have a normal st-link you can use the `probe-rs` tool to program it!

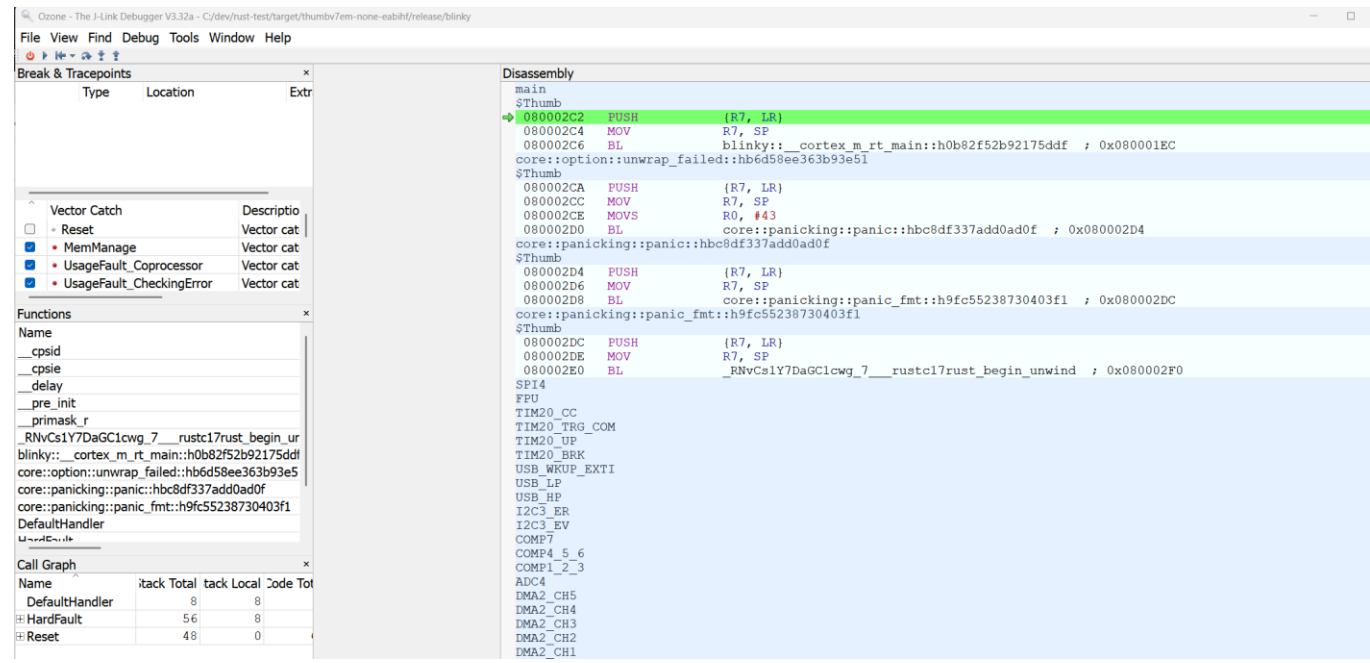
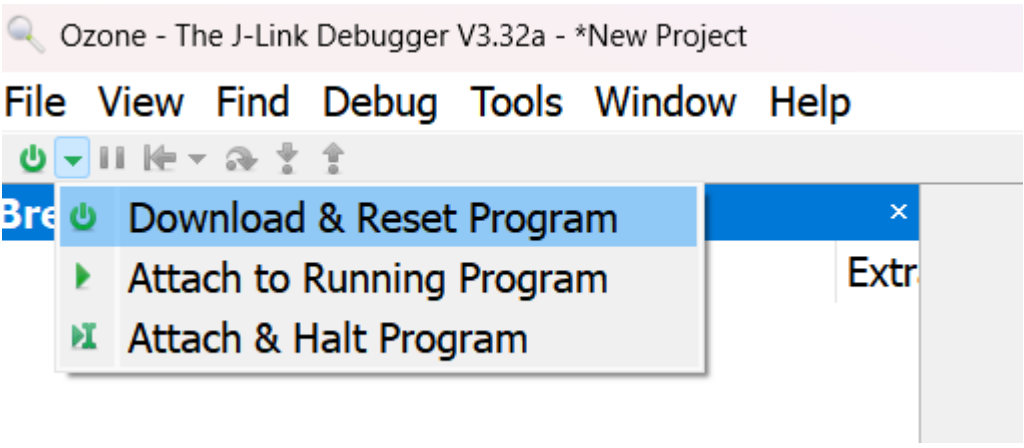
My Nucleo is a J-Link so have to do via Segger O-Zone



Set to “All Files” or rename the blinky to blinky.elf

Programming Blinky (Cont'd)

My Nucleo is a J-Link so have to do via Segger O-Zone



Example: More Complex Rust Program



```
let mut dp = pac::Peripherals::take().unwrap();
let cp = cortex_m::Peripherals::take().unwrap();

let mut flash = dp.FLASH.constrain();
let mut rcc = dp.RCC.constrain();
let mut syscfg = dp.SYSCFG.constrain(&mut rcc.apb2);
let mut gpioa = dp.GPIOA.split(&mut rcc.ahb);
let mut gpiob = dp.GPIOB.split(&mut rcc.ahb);
let mut gpioe = dp.GPIOE.split(&mut rcc.ahb);

// Initialize the system clock(s).
let clocks = rcc
    .cfgr
    .use_hse(8.MHz()) // STM32F3 Discovery has an 8 MHz
    .sysclk(48.MHz()) // Set system clock to 48 MHz.
```

<https://github.com/sunsided/stm32f3disco-rust/blob/main/src/main.rs>

More Rust Resources

Documentation & Books

- The Embedded Rust Book — book.rust-lang.org (free, official)
- Discovery Book — beginner projects for STM32 and micro:bit
- embassy.dev — Embassy framework docs with lots of examples
- rtic.rs — RTIC real-time concurrency framework docs

Get Started Today

- Pick a board: STM32 Nucleo, Raspberry Pi Pico, or BBC micro:bit
- awesome-embedded-rust on GitHub — curated list of crates
- Rust Embedded Matrix channel — Commonly referenced community to ask questions (*disclaimer: no personal experience*)
- embassy/examples/ on GitHub — Many more working code examples

Continuing Your Embedded Journey

- This is the end of Computer Architecture – hopefully you have a good foundation of *why* and *how* embedded systems work.
- If looking for more experience – the very low cost of boards like our STM32F3 Nucleo board and ESP32 boards can be a great starting point!